

CSCI 240 PA4 Submission

Exercise 1

Source code below:

```
package PA4;

import java.util.Stack;

public class PA4_Ex1 {

    public static void checkBalance(String exp) {
        Stack<String> stack = new Stack<>();
        String[] tokens = exp.split(" ");

        for (String token : tokens) {

            if (token.equals("(") || token.equals("[") ||
token.equals("<") || token.equals("{")) {
                stack.push(token);
            }
            else if (token.equals(")") || token.equals("]") ||
token.equals(">") || token.equals("}")) {
                if (stack.isEmpty()) {
                    System.out.println(exp + " \n    -> invalid, no
matching opening symbol");
                    return;
                }
                String open = stack.pop();
                if (!matches(open, token)) {
                    System.out.println(exp + " \n    -> invalid, closing
symbol does not match opening symbol");
                    return;
                }
            }
        }

        if (!stack.isEmpty()) {
            System.out.println(exp + " \n    -> invalid, no matching
closing symbol");
        } else {
            System.out.println(exp + " \n    -> valid");
        }
    }

    private static boolean matches(String open, String close) {
        return (open.equals("(") && close.equals(")") ||
                (open.equals("[") && close.equals("]")) ||
                (open.equals("<") && close.equals(">")) ||
                (open.equals("{") && close.equals("}")));
    }

    public static void main(String[] args) {
        System.out.println("Modified by: Aiden Wang\n");

        checkBalance("{ ( a + b ) * c1 }");
    }
}
```

```
        checkBalance("{ ( a + b ) * c1 ]");
        checkBalance("( < a + b > * c1 ) / 15 )");
        checkBalance("( ( ( a + b ) * c1 ) / 15 )");
        checkBalance("( < 12 % 3 > + 4 --");
    }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home
Modified by: Aiden Wang

{ ( a + b ) * c1 }
    -> valid
{ ( a + b ) * c1 ]
    -> invalid, closing symbol does not match opening symbol
( < a + b > * c1 ) / 15 )
    -> invalid, no matching opening symbol
( ( ( a + b ) * c1 ) / 15 )
    -> valid
( < 12 % 3 > + 4 --
    -> invalid, no matching closing symbol

Process finished with exit code 0
```

Exercise 2

Source code below:

```
package PA4;

public class PA4_Ex2 {

    static class MyArrayStack {
        private int[] data;
        private int t = -1;

        public MyArrayStack(int capacity) {
            data = new int[capacity];
        }
        public boolean isEmpty() { return t == -1; }
        public boolean isFull() { return t == data.length - 1; }

        public void push(int e) throws Exception {
            if (isFull()) throw new Exception("stack overflow");
            data[++t] = e;
        }
        public int pop() throws Exception {
            if (isEmpty()) throw new Exception("stack underflow");
            return data[t--];
        }
    }
}
```

```
    }  
}  
  
public static void evaluatePostfix(String exp) {  
  
    MyArrayStack stack = new MyArrayStack(5);  
    String[] tokens = exp.split(" ");  
  
    try {  
        for (String token : tokens) {  
            if (token.equals("+") || token.equals("-") ||  
token.equals("*") || token.equals("/")) {  
                int val2 = stack.pop();  
                int val1 = stack.pop();  
                int res = 0;  
  
                if (token.equals("+")) res = val1 + val2;  
                else if (token.equals("-")) res = val1 - val2;  
                else if (token.equals("*")) res = val1 * val2;  
                else if (token.equals("/")) res = val1 / val2;  
  
                stack.push(res);  
            } else {  
                stack.push(Integer.parseInt(token));  
            }  
            System.out.println(exp + " -> " + stack.pop());  
        } catch (Exception e) {  
            System.out.println(exp + " -> " + e.getMessage());  
        }  
    }  
  
public static void main(String[] args) {  
    System.out.println("Modified by: Aiden Wang\n");  
  
    evaluatePostfix("17 2 3 + / 13 -");  
    evaluatePostfix("5 2 3 + *");  
    evaluatePostfix("2 3 2 / *");  
    evaluatePostfix("-23 123 + 2 *");  
    evaluatePostfix("17 2 3 4 5 * 6 7 * + / + * -");  
}  
}
```

Due Date: 3/26/2026

Names: Aiden Wang

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Con
Modified by: Aiden Wang

17 2 3 + / 13 - -> -10
5 2 3 + * -> 25
2 3 2 / * -> 2
-23 123 + 2 * -> 200
17 2 3 4 5 * 6 7 * + / + * - -> stack overflow

Process finished with exit code 0
```

Exercise 3

Source code below:

```
package PA4;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Random;

public class PA4_Ex3 {

    static class Job {
        int id;
        int units;
        Job(int id, int units) {
            this.id = id;
            this.units = units;
        }
    }

    static class Node {
        Job data;
        Node next;
        Node(Job data) {
            this.data = data;
            this.next = null;
        }
    }

    static class MyLinkedList {
        private Node head = null;
        private Node tail = null;

        public void enqueue(Job j) {
            Node newNode = new Node(j);
            if (isEmpty()) {
                head = tail = newNode;
            }
        }
    }
}
```

```
        } else {
            tail.next = newNode;
            tail = newNode;
        }
    }

    public Job dequeue() {
        if (isEmpty()) return null;
        Job res = head.data;
        head = head.next;
        if (head == null) tail = null;
        return res;
    }

    public boolean isEmpty() {
        return head == null;
    }
}

public static void main(String[] args) {
    System.out.println("Modified by : Aiden Wang\n");

    System.out.println("Using MyLinkedListQueue with 2, 1, 2, 1, 1.");

    MyLinkedListQueue myQueue = new MyLinkedListQueue();
    int[] hardcoded = {2, 1, 2, 1, 1};

    for (int i = 0; i < 5; i++) {
        myQueue.enqueue(new Job(i + 1, hardcoded[i]));

        System.out.println("Enqueue job " + (i + 1) + " with " +
hardcoded[i] + " units");
    }

    System.out.println();

    int cycle = 1;
    while (!myQueue.isEmpty()) {
        Job currentJob = myQueue.dequeue();

        System.out.println(cycle + " Processing job " +
currentJob.id);
        currentJob.units--;

        if (currentJob.units > 0) {
            myQueue.enqueue(currentJob);
        } else {
            System.out.println("Done with job " + currentJob.id);
        }
        cycle++;
    }

    System.out.println("\n-----\n");

    System.out.println("Using Java LinkedList Queue with random
numbers.");
}
```

```

Queue<Job> javaQueue = new LinkedList<>();
Random rand = new Random();

for (int i = 0; i < 5; i++) {
    int randomUnits = rand.nextInt(5) + 1; // Generate random
units (1 to 5)
    javaQueue.add(new Job(i + 1, randomUnits));
    System.out.println("Enqueue job " + (i + 1) + " with " +
randomUnits + " units");
}

System.out.println();
cycle = 1;
while (!javaQueue.isEmpty()) {
    Job currentJob = javaQueue.remove();
    System.out.println( cycle + " Processing job " +
currentJob.id);

    currentJob.units--;
    if (currentJob.units > 0) {
        javaQueue.add(currentJob);
    } else {
        System.out.println("Done with job " + currentJob.id);
    }
    cycle++;
}
}

```

Input/output below:

```

/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Conte
Modified by : Aiden Wang

```

```

Using MyLinkedList with 2, 1, 2, 1, 1.

```

```

Enqueue job 1 with 2 units
Enqueue job 2 with 1 units
Enqueue job 3 with 2 units
Enqueue job 4 with 1 units
Enqueue job 5 with 1 units

```

```

1 Processing job 1
2 Processing job 2
Done with job 2
3 Processing job 3
4 Processing job 4
Done with job 4
5 Processing job 5
Done with job 5
6 Processing job 1
Done with job 1
7 Processing job 3
Done with job 3

```

```

-----

```

```
Modified by : Aiden Wang

Using Java LinkedList Queue with random numbers.
Enqueue job 1 with 2 units
Enqueue job 2 with 1 units
Enqueue job 3 with 1 units
Enqueue job 4 with 4 units
Enqueue job 5 with 1 units

1 Processing job 1
2 Processing job 2
Done with job 2
3 Processing job 3
Done with job 3
4 Processing job 4
5 Processing job 5
Done with job 5
6 Processing job 1
Done with job 1
7 Processing job 4
8 Processing job 4
9 Processing job 4
Done with job 4

Process finished with exit code 0
```

Answer for Question 1

Question: Given a choice between an array and a linked list, which one is better to implement a queue? Explain.

Answer: The better choice depends entirely on whether the maximum capacity of the queue is known in advance.

When an Array is better: If the maximum size is known, an array is highly recommended, provided it is implemented as a circular array. By using variables to track the front index and current size, you can use modular arithmetic to wrap around the array's ends. This ensures both enqueue and dequeue operations run in extremely fast $O(1)$ time. Arrays are also more memory-efficient and practically faster because they avoid the overhead of instantiating new node objects and storing extra reference pointers.

When a Linked List is better: If the queue's size is unknown, unpredictable, or needs to grow massively, a singly linked list is the better choice. It grows

dynamically, guaranteeing a strict $O(1)$ worst-case time for all operations because it never hits an artificial capacity limit and never has to waste time resizing itself. The trade-off is slightly higher memory usage per element and slightly slower practical runtimes due to the continuous creation and relinking of nodes.

Answer for Question 2

Question: Would you consider an array implementation for the deque? Explain why or why not.

Answer: Yes, an array implementation is an excellent choice for a deque, again with the strict requirement that it is treated as a circular array.

Why it works: By tracking the front index and the current size, a circular array allows you to seamlessly wrap around the bounds of the fixed-capacity structure. This allows all four core update methods of a deque (addFirst, addLast, removeFirst, and removeLast) to execute in perfect $O(1)$ constant time.

The implementation catch: Because a deque allows you to add elements to the front of the list, the front index must frequently step backwards. When implementing the modular arithmetic, you must ensure that moving backwards from index 0 properly wraps around to the very end of the array instead of producing an invalid negative index (e.g., using a formula like $(f - 1 + \text{capacity}) \% \text{capacity}$).

Extra Credit – Option 2: MyLinkedDeque

Source code below:

```
package PA4;

public class PA4_EC2 {

    static class Node {
        String data;
        Node prev;
        Node next;

        Node(String data) {
            this.data = data;
        }
    }

    // Custom Linked Deque Implementation
    static class MyLinkedDeque {
        private Node header; // Sentinel node at the front
        private Node trailer; // Sentinel node at the rear
        private int size = 0;

        public MyLinkedDeque() {
            header = new Node(null);
            trailer = new Node(null);
            header.next = trailer;
        }
    }
}
```

```
        trailer.prev = header;
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public void insertFront(String e) {
        Node newNode = new Node(e);
        newNode.next = header.next;
        newNode.prev = header;
        header.next.prev = newNode;
        header.next = newNode;
        size++;
    }

    public void insertRear(String e) {
        Node newNode = new Node(e);
        newNode.prev = trailer.prev;
        newNode.next = trailer;
        trailer.prev.next = newNode;
        trailer.prev = newNode;
        size++;
    }

    public String removeFront() {
        if (isEmpty()) return null;
        Node first = header.next;
        String res = first.data;
        header.next = first.next;
        first.next.prev = header;
        size--;
        return res;
    }

    public String removeRear() {
        if (isEmpty()) return null;
        Node last = trailer.prev;
        String res = last.data;
        trailer.prev = last.prev;
        last.prev.next = trailer;
        size--;
        return res;
    }

    public String getFront() {
        if (isEmpty()) return null;
        return header.next.data;
    }

    public void printDeque() {
        System.out.print(" -> Current Deque: [");
        Node curr = header.next;
```

```
        while (curr != trailer) {
            System.out.print(curr.data);
            curr = curr.next;
            if (curr != trailer) {
                System.out.print(", ");
            }
        }
        System.out.println("]");
    }
}

public static void main(String[] args) {
    System.out.println("Modified by : Aiden Wang\n");

    MyLinkedDeque deque = new MyLinkedDeque();

    // Required Test Cases [2]
    System.out.println("Action: insert front (\"John\")");
    deque.insertFront("John");
    deque.printDeque();

    System.out.println("\nAction: insert front (\"Jane\")");
    deque.insertFront("Jane");
    deque.printDeque();

    System.out.println("\nAction: insert rear (\"Jo\")");
    deque.insertRear("Jo");
    deque.printDeque();

    System.out.println("\nAction: remove rear");
    String removedRear = deque.removeRear();
    System.out.println(" -> Removed: " + removedRear);
    deque.printDeque();

    System.out.println("\nAction: remove front");
    String removedFront = deque.removeFront();
    System.out.println(" -> Removed: " + removedFront);
    deque.printDeque();

    System.out.println("\nAction: output size");
    System.out.println(" -> Size: " + deque.size());

    System.out.println("\nAction: output front item");
    System.out.println(" -> Front item: " + deque.getFront());
}
}
```

Input/output below:

Due Date: 3/26/2026

Names: Aiden Wang

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/  
Modified by : Aiden Wang  
  
Action: insert front ("John")  
-> Current Deque: [John]  
  
Action: insert front ("Jane")  
-> Current Deque: [Jane, John]  
  
Action: insert rear ("Jo")  
-> Current Deque: [Jane, John, Jo]  
  
Action: remove rear  
-> Removed: Jo  
-> Current Deque: [Jane, John]  
  
Action: remove front  
-> Removed: Jane  
-> Current Deque: [John]  
  
Action: output size  
-> Size: 1  
  
Action: output front item  
-> Front item: John  
  
Process finished with exit code 0
```